ВЫЧИСЛИТЕЛЬНЫЕ МЕТОДЫ · ЛЕКЦИЯ 6

Метод Монте-Карло в задачах переноса

От случайного блуждания к уравнению Больцмана и фононному переносу

Лю Шисян

Кафедра теплофизики (Э6)
МГТУ им. Н.Э. Баумана
Москва / 2026

Место этой лекции в курсе



Лекция 6 — Monte Carlo в задачах переноса

Рассмотрим движение частиц между столкновениями, рассеяние и оценку температуры и теплового потока по ансамблю траекторий.

Задачи занятия:

- моделировать свободный полёт и случайные события;
- объяснять связь ансамбля траекторий с уравнением Больцмана;
- описывать базовый алгоритм фононного Monte Carlo.

План

1. Модели переноса частиц
2. Метод Monte Carlo в задачах переноса
3. Уравнение Больцмана и фононный Monte Carlo
4. Практика и итоги

1

Модели переноса частиц

Случайное блуждание и транспортный Монте-Карло

В лекции 5 мы рассматривали простое случайное блуждание:

$$x_{n+1} = x_n + \Delta x_n, \qquad \Delta x_n = \pm \ell.$$

Случайное блуждание

$$\Delta t = \text{const}, \qquad |\Delta x| = \ell.$$

На каждом шаге:

$$x_n \xrightarrow{\text{случайный выбор}} x_{n+1}.$$

Случайность вводится **на каждом временном шаге**.

Transport Monte Carlo

Частица сначала движется свободно:

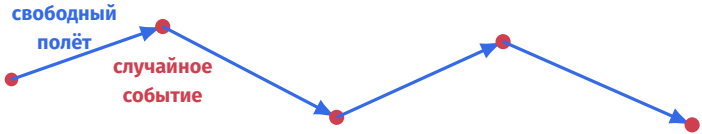
$$\mathbf{r}(t+\tau) = \mathbf{r}(t) + \mathbf{v}\,\tau,$$

а затем происходит случайное событие.
Случайными могут быть:

- время до следующего события;
- тип происходящего события;
- состояние частицы после события.

Микроскопическая картина переноса

Движение состоит из свободных полётов и случайных событий.



Свободный полёт

При заданной длительности полёта h движение детерминировано:

$$\mathbf{r}_{new} = \mathbf{r}_{old} + \mathbf{v}h.$$

Событие

По заданным вероятностям выбираем новое направление, скорость или энергию.

В нашей gray-модели изменяется только направление.

Состояние вычислительной частицы

Чтобы продолжить траекторию, программа должна хранить **текущее состояние** частицы.

$$\mathcal{S} = \{\mathbf{r},\, \mathbf{v},\, E,\, t,\, w,\, s, \ldots\}$$

Кинематические переменные

$\mathbf{r}$ положение,

$\mathbf{v}$ скорость и направление,

t текущее время.

Они определяют движение частицы между событиями.

Внутреннее состояние

E энергия,

s тип, мода, ветвь и т.п.,

w статистический вес.

Они определяют вероятности последующих процессов.

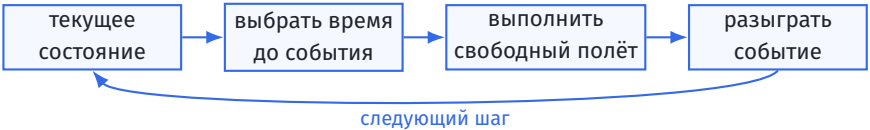
Вычислительная частица

Одна Monte Carlo-частица не обязана соответствовать одной физической частице. При весе w одна траектория может представлять большое число одинаковых физических носителей.

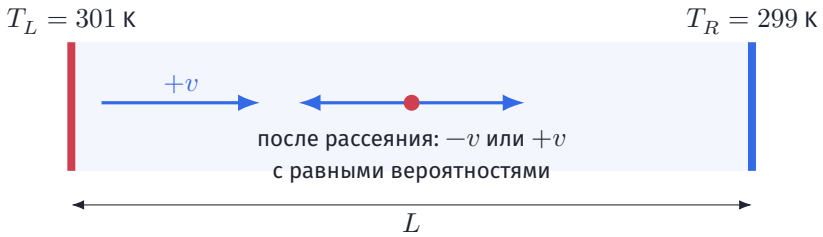
Носители и события в задачах переноса

Носитель и набор событий зависят от физической задачи.

Задача	Носитель	Движение	Случайные события
Нейтронный перенос	нейтрон	свободный полёт	рассеяние, поглощение
Перенос излучения	фотон	распространение в среде	рассеяние, поглощение
Полупроводники	электрон	движение в поле	рассеяние на фононах, примесях, дефектах
Теплоперенос	фонон	свободный полёт с групповой скоростью	рассеяние на фононах, дефектах, электронах и границах



Учебная модель: слой и две скорости



- Границы поглощают выходящие частицы.
- v, τ и теплоёмкость C постоянны. Энергия пакета сохраняется.
- Направление после рассеяния может совпасть с прежним.

Единицы в расчёте

Безразмерные переменные

$$x^* = \frac{x}{v\tau}, \qquad t^* = \frac{t}{\tau}, \qquad L^* = \frac{L}{v\tau}.$$

Скорость равна $+1$ или -1.

Среднее время между событиями равно 1.

В коде `length` означает L^*.

Пример в единицах СИ

$$v = 1000 \text{ м/с}, \qquad \tau = 10^{-10} \text{ с}.$$

$$v\tau = 100 \text{ нм}, \qquad L^* = 5.$$

Толщина слоя: $L = 500$ нм.

Это параметры учебной модели, без спектра реального материала.

2

Метод Monte Carlo в задачах переноса

Распределение времени свободного полёта

Пусть случайные события происходят с постоянной интенсивностью

$$\Gamma = \frac{1}{\tau},$$

где Γ — интенсивность событий, а τ — среднее время между событиями.

Вероятность свободного полёта

Вероятность того, что за время t событие ещё не произошло:

$$\boxed{P_{\text{free}}(t) = e^{-\Gamma t} = e^{-t/\tau}}$$

Среднее время свободного полёта

Для экспоненциального закона среднее время до события равно

$$\boxed{\langle t_{\text{free}} \rangle = \frac{1}{\Gamma} = \tau}$$

Генерация времени свободного полёта

Вероятность отсутствия события:

$$P_{\text{free}}(t) = e^{-t/\tau}.$$

Генерируем случайное число

$$\xi \in (0, 1)$$

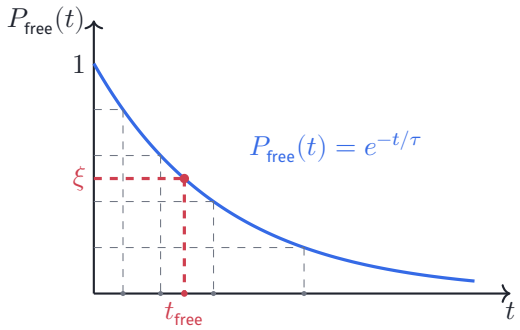
и приравниваем:

$$\xi = P_{\text{free}}(t_{\text{free}}) = e^{-t_{\text{free}}/\tau}.$$

Отсюда

$$\ln \xi = -\frac{t_{\text{free}}}{\tau},$$

$$t_{\text{free}} = -\tau \ln \xi$$



Время свободного полёта в NumPy

```
import numpy as np

rng = np.random.default_rng(42)
tau = 2.0
N = 100_000
u = rng.random(N)
flight = -tau * np.log1p(-u)
flight_builtin = rng.exponential(tau, N)
```

`exponential` принимает среднее время, а не частоту событий.

Обратное преобразование

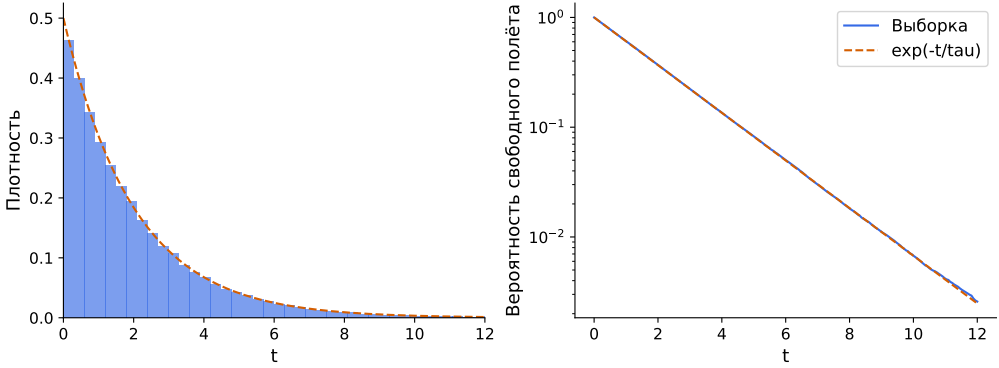
$$t_{\text{free}} = -\tau \ln(1-u).$$

`random` возвращает $u \in [0, 1)$.

`log1p(-u)` вычисляет $\ln(1-u)$.

При $\tau = 2$ среднее выборки: 2.00280.

Проверка распределения свободного полёта



100 000 времён свободного полёта, $\tau = 2$.

Справа: вероятность не встретить событие до момента t.

Что произойдёт раньше: рассеяние или граница?

Два времени

$$t_s \sim \mathrm{Exp}(\tau), \qquad t_b = \frac{d_b}{v}.$$

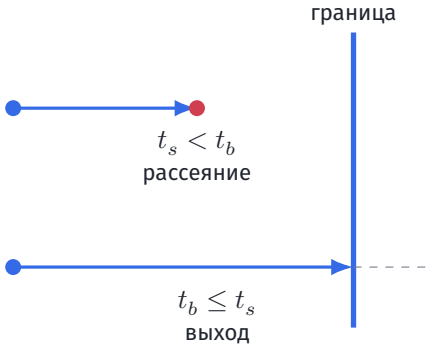
d_b: расстояние до границы по направлению движения.

$$h = \min(t_s, t_b).$$

Перемещаем частицу на $v_x h$.

Если $t_b \leq t_s$, частица выходит из слоя.

Иначе разыгрываем новое направление.



Выбор типа события

Пусть возможны несколько независимых процессов с интенсивностями $\Gamma_1, \Gamma_2, \ldots, \Gamma_N$. Общая интенсивность событий:

$$\Gamma_{\text{tot}} = \Gamma_1 + \Gamma_2 + \cdots + \Gamma_N$$

и среднее время до следующего события:

$$\tau = \frac{1}{\Gamma_{\text{tot}}}$$

Вероятность каждого события

Вероятность того, что произойдёт событие типа i:

$$P_i = \frac{\Gamma_i}{\Gamma_{\text{tot}}}$$

Розыгрыш события

Генерируем новое случайное число $\xi_2 \in (0, 1)$. Например, для трёх процессов:

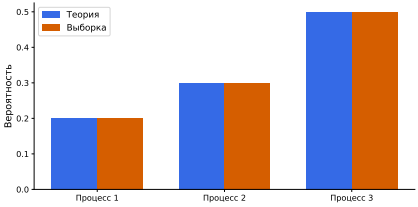
$$0 < \xi_2 < P_1 \quad \Rightarrow \quad \text{событие 1},$$

$$P_1 < \xi_2 < P_1 + P_2 \quad \Rightarrow \quad \text{событие 2},$$

$$P_1 + P_2 < \xi_2 < 1 \quad \Rightarrow \quad \text{событие 3}.$$

Выбор процесса на Python

```
rates = np.array([0.2, 0.3, 0.5])
probabilities = rates / rates.sum()
event_time = rng.exponential(
    1 / rates.sum(), N
)
event_type = rng.choice(
    3, size=N, p=probabilities
)
```



Для каждого события независимо выбираем время и тип.

Состояние частицы после события

После выбора типа события необходимо определить, как это событие изменяет состояние частицы.

$$\mathcal{S}_{\text{old}} \longrightarrow \mathcal{S}_{\text{new}}$$

Изменяемые параметры

В зависимости от типа события могут измениться:

- направление движения;
- скорость;
- энергия;
- внутреннее состояние.

Розыгрыш нового состояния

Для выбранного события разыгрывается его конкретный результат.
Например:

- при рассеянии на границе — новое направление;
- для фонона — новая мода или ветвь.

Одна событийная траектория

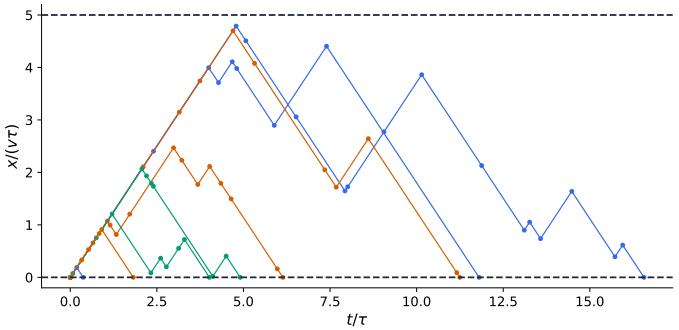
```
def single_path(length, rng):
    x, t, direction = 0.0, 0.0, 1.0
    positions, times = [x], [t]
    while True:
        ts = rng.exponential(1.0)
        tb = length - x if direction > 0 else x
        h = min(ts, tb)
        x = np.clip(x + direction * h, 0, length)
        t += h
        positions.append(x)
        times.append(t)
        if tb <= ts:
            break
        direction = rng.choice([-1.0, 1.0])
    return np.array(times), np.array(positions)
```

Вход слева: $x = 0$, направление $+1$.

Расчёт заканчивается на одной из границ.

Время в единицах τ, координата в единицах $v\tau$.

Траектории в слое



8 частиц, $L^* = 5$

Часть частиц проходит слой, часть возвращается в левый резервуар.

Точки отмечают события и выход.

После события наклон может остаться прежним.

Вероятность прохождения слоя

```
rng = np.random.default_rng(2026)
N = 100_000
length = 5.0
result = event_transport(N, length, rng)
p_mc = result["transmitted"].mean()
se = np.sqrt(p_mc * (1 - p_mc) / N)
p_exact = 1 / (1 + length / 2)
```

`event_transport` применяет тот же порядок событий к массиву активных частиц.

100 000 траекторий

$$P_{MC} = 0.28435.$$

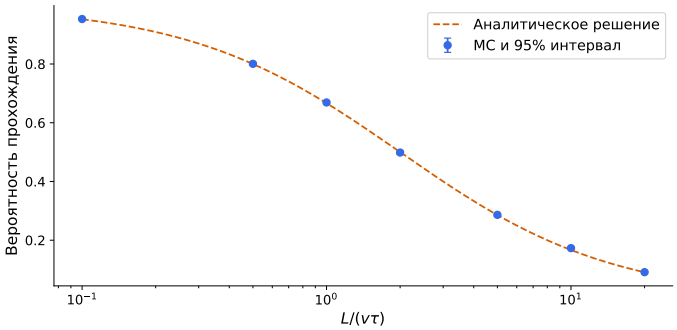
$$SE = 0.00143.$$

Аналитическая проверка

$$P_{tr} = \frac{1}{1 + L/(2v\tau)}.$$

Для $L^* = 5$: $P_{tr} = 0.28571.$

Толщина слоя и прохождение



По 30 000 частиц для каждой толщины.

При $L \ll v\tau$ большинство частиц проходит без рассеяния.

С ростом толщины всё больше частиц возвращается к входу.

Событийный и пошаговый методы

Событийный Monte Carlo

Сразу генерируется время до следующего события:

$$t_{\text{free}}^{(n)} = -\tau \ln \xi_n.$$

После этого время обновляется:

$$\boxed{t_{n+1} = t_n + t_{\text{free}}^{(n)}}$$

Интервалы между событиями **случайны**.

Пошаговый Monte Carlo

Заранее задаётся постоянный шаг

$$\Delta t = \text{const}.$$

Время обновляется по правилу:

$$\boxed{t_{n+1} = t_n + \Delta t}$$

На каждом шаге проверяется, произошло событие или нет.
Шаг по времени **фиксирован**.

Пошаговый Monte Carlo

В пошаговом Monte Carlo движение системы разбивается на одинаковые интервалы времени

$$\Delta t = \text{const}$$

и расчёт последовательно выполняется для каждого временного шага.

1. Перемещение

За один временной шаг частица перемещается на расстояние

$$\Delta \mathbf{r} = \mathbf{v}\,\Delta t.$$

Поэтому

$$\mathbf{r}_{n+1} = \mathbf{r}_n + \mathbf{v}_n \Delta t$$

и

$$t_{n+1} = t_n + \Delta t.$$

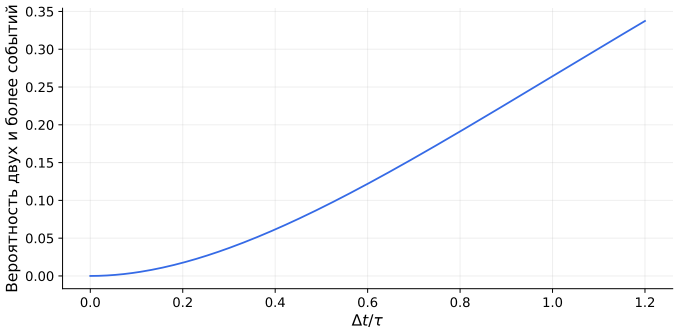
2. Проверка события

За время Δt событие может произойти с некоторой вероятностью P_{event}.
Генерируем случайное число $\xi \in (0, 1)$.
Если

$$\xi < P_{\text{event}}$$

то событие происходит.
Если нет — частица продолжает движение.

Вероятность события и выбор шага



Хотя бы одно событие

$$P = 1 - e^{-\Delta t/\tau}.$$

Эта вероятность точна при постоянном τ.

Но проверка только в конце шага не восстанавливает моменты событий внутри него.

Нужна проверка сходимости по Δt.

Событийный и пошаговый Monte Carlo: сравнение

Два подхода отличаются тем, **как организовано продвижение системы во времени**.

	Событийный Monte Carlo	**Пошаговый Monte Carlo**
Основная идея	Частица сразу переходит от одного события к следующему	Все частицы продвигаются с одинаковым шагом Δt
Внешний цикл	Обычно по частицам и их траекториям	Обычно по времени
Главное преимущество	**Высокая эффективность:** не тратятся вычисления на шаги без событий	**Удобно учитывать эволюцию системы:** на каждом шаге можно обновлять локальные параметры
Ограничение	Сложнее при сильном изменении температуры и интенсивностей событий	Малый Δt увеличивает число шагов и вычислительные затраты

Пошаговый расчёт: движение и рассеяние

```
probability = -np.expm1(-dt)
ids = np.flatnonzero(alive)
tb = np.where(direction[ids] > 0,
              length - x[ids], x[ids])
hit = tb <= dt * (1 + 1e-12)
h = np.minimum(tb, dt)
x[ids] = np.clip(x[ids] + direction[ids] * h, 0, length)
transmitted[ids[hit]] = direction[ids[hit]] > 0
alive[ids[hit]] = False
remain = ids[~hit]
scatter = remain[
    rng.random(remain.size) < probability
]
direction[scatter] = rng.choice(
    [-1.0, 1.0], scatter.size
)
```

Тело цикла по времени

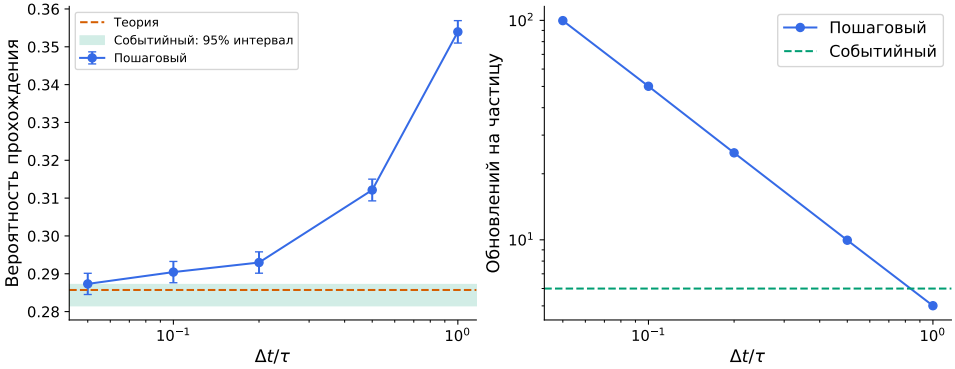
`alive`: частицы внутри слоя.

`hit`: выход на границу.

`scatter`: рассеяние в конце шага.

Здесь `dt` означает $\Delta t/\tau$.

Влияние шага: точность и число обновлений



$L/(v\tau) = 5$, по 100 000 частиц в каждом расчёте.

Уменьшение шага снижает смещение результата и увеличивает число обновлений.

Статистическая ошибка и ошибка шага

Больше частиц

Снижает статистический шум.

При $N = 100\,000$ стандартная ошибка вероятности составляет около 0.0014.

Меньше шаг

Снижает погрешность пошагового алгоритма.

При $\Delta t/\tau = 1$: $P_{MC} = 0.35395$.

При $\Delta t/\tau = 0.05$: $P_{MC} = 0.28732$.

Аналитическое значение: 0.28571.

Если время рассеяния зависит от температуры

На каждом временном шаге

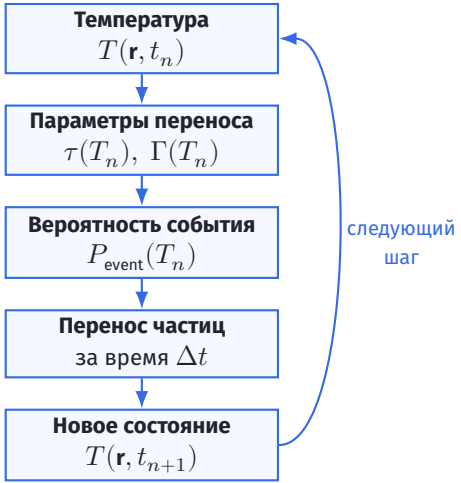
Для момента t_n:

1. определяем локальную температуру $T(\mathbf{r}, t_n)$;
2. обновляем

$$\tau(T_n), \qquad \Gamma(T_n);$$

3. определяем вероятность события;
4. перемещаем частицы за Δt;
5. рассчитываем новое состояние системы

$$T(\mathbf{r}, t_{n+1}).$$



3

Уравнение Больцмана и фононный Monte Carlo

Уравнение Больцмана для фононов

Для фононной моды λ функция распределения $f_\lambda(\mathbf{r}, t)$ показывает, насколько заполнено данное состояние в точке **r** в момент времени t.
Уравнение переноса Больцмана имеет вид

$$\frac{\partial f_\lambda}{\partial t} + \mathbf{v}_\lambda \cdot \nabla_\mathbf{r} f_\lambda = \left(\frac{\partial f_\lambda}{\partial t}\right)_{\text{coll}}$$

Перенос

$$\frac{\partial f_\lambda}{\partial t} + \mathbf{v}_\lambda \cdot \nabla_\mathbf{r} f_\lambda$$

описывает изменение распределения из-за движения фононов с групповой скоростью $\mathbf{v}_\lambda$.

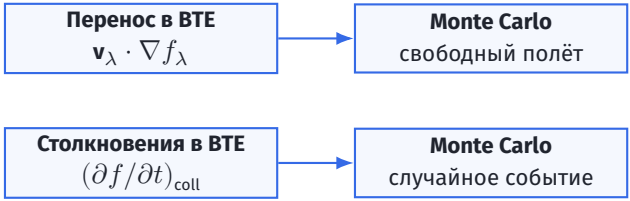
Столкновения

$$\left(\frac{\partial f_\lambda}{\partial t}\right)_{\text{coll}}$$

описывает изменение распределения из-за процессов рассеяния.
При рассеянии фонон может перейти $\lambda \to \lambda'$.

Траектории и функция распределения

Уравнение Больцмана описывает **распределение**, а Monte Carlo моделирует **отдельные траектории**.



Для одной частицы:

$$\text{свободный полёт} \rightarrow \text{рассеяние} \rightarrow \text{свободный полёт} \rightarrow \cdots$$

Для большого ансамбля частиц:

$$\text{множество траекторий} \rightarrow f_\lambda(\mathbf{r}, t)$$

Фонон как вычислительная частица

Фононное состояние обозначим

$$\boxed{\lambda = (\mathbf{q}, s)}$$

где **q** — волновой вектор, а s — номер фононной ветви.

Свойства фононной моды

Для каждой моды λ определены

ω_λ частота,

$\mathbf{v}_\lambda$ групповая скорость,

τ_λ время жизни,

$\varepsilon_\lambda = \hbar\omega_\lambda$ энергия.

Состояние МС-частицы

Вычислительная частица может хранить

$$\boxed{\mathcal{S} = \{\mathbf{r}, \lambda, t, w\}}$$

где

r — положение,

λ — фононная мода,

w — статистический вес.

Событийный и пошаговый фононный Монте-Карло

Событийный Monte Carlo

Для текущей моды λ

$$t_{\text{scat}} = -\tau_\lambda \ln \xi$$

и фонон движется со скоростью

$$\mathbf{v}_\lambda.$$

Положение обновляется:

$$\mathbf{r}' = \mathbf{r} + \mathbf{v}_\lambda t_{\text{scat}}.$$

Пошаговый Monte Carlo

Все фононы продвигаются на

$$\Delta t = \text{const}.$$

Для моды λ

$$P_{\text{scat}} = 1 - e^{-\Delta t/\tau_\lambda}$$

или, если свойства зависят от температуры,

$$\tau_\lambda = \tau_\lambda(T).$$

Gray-модель: одно τ. Full-band: своё τ_λ для каждой моды.

Оценка температуры и теплового потока

Энергия и температура

В ячейке объёма V_c:

$$u_c = \frac{1}{V_c} \sum_{i \in c} w_i \hbar \omega_i .$$

Температура определяется из

$$u_c = u_{\text{eq}}(T_c).$$

$$u_{\text{eq}}(T) = \frac{1}{V} \sum_{\lambda} \hbar \omega_\lambda f^0_\lambda(T).$$

Тепловой поток

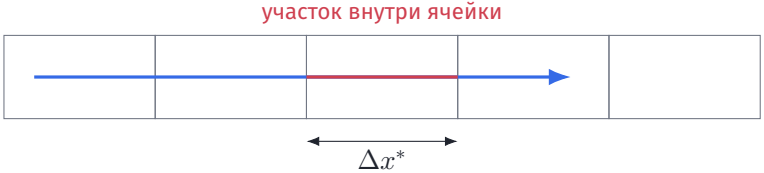
Энергия переносится со скоростью $\mathbf{v}_i$:

$$\mathbf{q}_c \approx \frac{1}{V_c} \sum_{i \in c} w_i \hbar \omega_i \mathbf{v}_i .$$

w_i — число физических фононов, представляемых частицей. Оценки усредняют по времени и независимым запускам.

Температура из времени пребывания

Равновесный фон $T_R = 299$ K учитываем точно. Частицы переносят дополнительную энергию, связанную с $\Delta T = 2$ K.



r_j: среднее время в ячейке по всем вошедшим частицам, в единицах τ.

$$T_j = T_R + \frac{\Delta T}{2\Delta x^*} r_j.$$

Температура и поток в коде

```
result = event_transport(
    N_batch, length, rng, n_cells=n_cells
)
T_batches.append(
    TR + DeltaT * result["residence"] / (2 * dx)
)
q_batches.append(
    q_ballistic * result["signed_residence"] / dx
)
p_batches.append(result["transmitted"].mean())
```

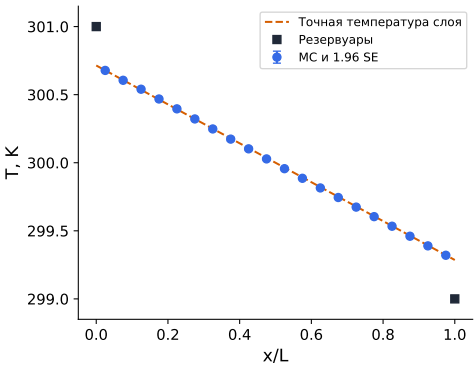
Один из 20 запусков

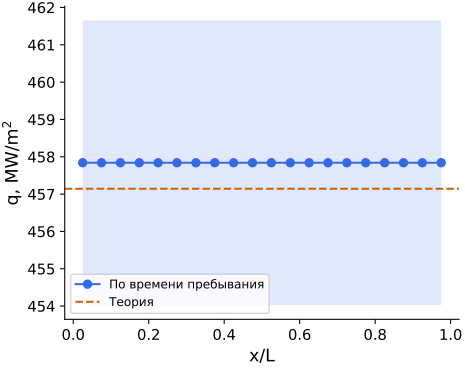
По 4000 частиц, 20 пространственных ячеек.

`residence`: время пребывания.

`signed_residence`: время со знаком направления.

Температура слоя и стационарный поток





Температура слоя у границы отличается от температуры резервуара.

Тепловой поток одинаков во всех ячейках. Полосы показывают $1.96SE$.

Поток и эффективная теплопроводность

```
C = 1.6e6
v = 1000.0
tau = 1e-10
TL, TR = 301.0, 299.0
DeltaT = TL - TR
mean_free_path = v * tau
length = 5.0
L = length * mean_free_path
k_bulk = C * v**2 * tau
q_ballistic = C * v * DeltaT / 2
q_mc = q_ballistic * p_batches.mean()
k_eff = q_mc * L / DeltaT
```

C в Дж/(м^3 К), v в м/с, τ в с.

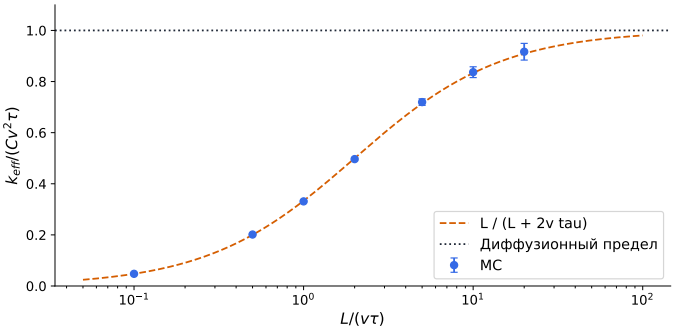
Результат расчёта

$$q_{MC} = 457.84 \text{ МВт/м}^2.$$

$$SE_q = 1.95 \text{ МВт/м}^2.$$

$$k_{eff} = 114.46 \text{ Вт/(м К)}.$$

От баллистического переноса к диффузии



Две скорости $\pm v$

$$k_{bulk} = Cv^2\tau.$$

$$\frac{k_{eff}}{k_{bulk}} = \frac{L}{L + 2v\tau}.$$

При $L \gg v\tau$ получаем диффузионный предел.

Коэффициент $1/3$ для трёхмерной изотропной модели здесь не применяется.

4

Практика и итоги

Практическое задание

Прохождение слоя

- Измените $L/(v\tau)$. Сравните долю прошедших частиц с $1/(1 + L/(2v\tau))$.

Пошаговый метод

- Сначала уменьшите Δt, затем увеличьте число частиц.
- Разделите статистический шум и погрешность временного шага.

Тепловой поток

- Измените ΔT. Проверьте пропорциональность q при постоянных свойствах.

Итоги лекции

- Время свободного полёта определяется частотой событий.
- При движении нужно сравнивать время до рассеяния и до границы.
- Пошаговый метод проверяют по числу частиц и по временному шагу.
- По прохождению и времени пребывания получаем поток и температуру слоя.

github.com/lyushisyan/computational-methods-course